Repaso PHP · DWES — examen 19 de octubre

2º DAW · DWES · Basado en el repositorio Daw2Decroly2027 (DWES (BACK)/PHP)

1. Mapa de lo que has dado

Carpeta	Tema	Qué practicas
php1Actividad / a1	Variables y tipos	variables, concatenación, operadores, gettype, == vs ===
php1Actividad / a2	Condicionales básicos	if / else, && y ||
php2Actividad / a1	Condicionales avanzados	módulo %, switch, ternario
php2Actividad / a2	Bucles	for, while, do-while, foreach
php2Actividad / a3	Arrays	indexados, asociativos, multidimensionales
php3Actividad / a1	Funciones	parámetros, valor por defecto, return, global
php3Actividad / a2	Formularios	GET, POST, isset, empty, trim, htmlspecialchars
php4Actividad / a1	JSON y validación	file_get_contents, json_decode, filter_var, sanitizar
php4Actividad / a2	Cadenas	str_replace, substr

Cada bloque recoge **cómo funciona** el tema y el **código correcto** de las actividades del repositorio. Al final hay ejercicios de examen resueltos, una chuleta y un plan de repaso.

2. Variables, tipos y operadores (php1 a1 y a2)

Cómo funciona

- Las variables empiezan por `$`, no se declaran con tipo y distinguen mayúsculas (`$Nombre` ≠ `$nombre`).

- `.` concatena texto; `+` suma números. `"Hola " . $nombre` ✔ — `"Hola " + $nombre` ✘.
- Tipos: `string`, `int`, `float`, `bool`, `array`, `null`. Se consultan con `gettype($x)` o `var_dump($x)`.
- `==` compara valor; `===` compara valor **y** tipo. `"8" == 8` → true, `"8" === 8` → false.
- Operadores: `+ - * / % **` (potencia), `&& || !` (lógicos), `??` (valor por defecto si no existe), `<=>` (nave espacial).
- Un entero demasiado grande pasa a `float` (por ejemplo, `101010101110101010000` se imprime en notación científica).

```
$nombre = "Juan";
$apellidos = "Pérez";
$edad = 20;
echo "Hola $nombre $apellidos, con edad: " . $edad . "<br>";

$a = 5; $b = 10;
echo "Suma: " . ($a + $b) . "<br>";
echo "Resta: " . ($a - $b) . "<br>";
echo "Multiplicación: " . ($a * $b) . "<br>";
echo "División: " . ($a / $b) . "<br>";

echo gettype("Hola") . "<br>";  // string
echo gettype(1) . "<br>";       // integer
echo gettype(1.0) . "<br>";     // double (el float de PHP)
echo gettype(false) . "<br>";   // boolean

var_dump("8" == 8);   // bool(true): compara solo el valor
var_dump("8" === 8);  // bool(false): compara valor y tipo

$edad = 20;
$carnet = true;
echo ($edad >= 18 && $carnet) ? "Puedes conducir" : "No puedes conducir";
```

Ejemplo resuelto: media de tres notas (aprobado si la media es ⩾ 5 y ninguna nota es menor que 4)

```
$n1 = 6; $n2 = 3; $n3 = 8;
$media = ($n1 + $n2 + $n3) / 3;
if ($media >= 5 && $n1 >= 4 && $n2 >= 4 && $n3 >= 4) {
    echo "Aprobado";
} else {
    echo "Suspenso";
}
```

3. Condicionales (php1 a2 y php2 a1)

Cómo funciona

- `if / elseif / else` con condiciones combinadas con `&&` (y), `||` (o), `!` (no).
- `%` devuelve el resto: `$n % 2 == 0` → par.
- `switch` compara una variable con cada `case` usando `==`. **Cada case necesita `break`** o el código «cae» al siguiente.
- Ternario: `$x = condicion ? "si" : "no";`
- Para rangos en un switch se usa el truco `switch (true)`.

```
$numero = 7;
echo ($numero % 2 == 0) ? "par" : "impar";

$dia = 3;
switch ($dia) {
    case 1: echo "Lunes"; break;
    case 2: echo "Martes"; break;
    case 3: echo "Miércoles"; break;
    default: echo "Del 1 al 7";
}

$nota = 7;
switch (true) {
    case $nota < 5:  echo "Suspenso"; break;
    case $nota == 5: echo "Suficiente"; break;
    case $nota == 6: echo "Bien"; break;
    case $nota <= 8: echo "Notable"; break;
    default:         echo "Sobresaliente";
}
```

4. Bucles (php2 a2)

Cómo funciona

- `for (inicio; condición; paso)` cuando sabes cuántas vueltas.
- `while (condición)` comprueba **antes** de cada vuelta; `do { } while (condición);` ejecuta **al menos una vez**.
- `foreach ($array as $valor)` o `foreach ($array as $clave => $valor)` para recorrer arrays.
- `break` sale del bucle; `continue` salta a la siguiente vuelta.
- Un `while` cuya condición nunca cambia es un bucle infinito.

```php
// Tablas de multiplicar del 1 al 10
for ($i = 1; $i <= 10; $i++) {
    echo "<h3>Tabla del $i</h3>";
    for ($j = 1; $j <= 10; $j++) {
        echo "$i x $j = " . ($i * $j) . "<br>";
    }
}

// Pares del 0 al 50
for ($i = 0; $i <= 50; $i += 2) { echo $i . " "; }

// Cuenta atrás con do-while
$n = 10;
do { echo $n . " "; $n--; } while ($n >= 0);

// IVA del 21% a un array de precios
$precios = [12.5, 8, 25.3, 4.99, 15];
foreach ($precios as $precio) {
    echo $precio . " € → " . round($precio * 1.21, 2) . " €<br>";
}
```

5. Arrays (php2 a3)

Cómo funciona

- **Indexado**: `$a = ["Leche", "Huevos"];` → `$a[0]`.
- **Asociativo**: `$p = ["nombre" => "Leche", "precio" => 1.5];` → `$p["nombre"]`.
- **Multidimensional**: array de arrays asociativos, el patrón más típico de examen (lista de productos, alumnos, usuarios).
- Añadir: `$a[] = "Pan";` o `array_push`. Contar: `count($a)`.
- Recorrer con `foreach`; el índice sale con `$indice => $valor`.

```
$productos = [
    ["nombre" => "Leche", "precio" => 1.50, "categoria" => "Lacteos"],
    ["nombre" => "Queso", "precio" => 2.20, "categoria" => "Lacteos"],
    ["nombre" => "Pan",   "precio" => 1.10, "categoria" => "Panaderia"],
];

$total = 0;
foreach ($productos as $p) {
    if ($p["categoria"] == "Lacteos") {
        echo $p["nombre"] . " - " . $p["precio"] . " €<br>";
        $total += $p["precio"];
    }
}
echo "Total: " . $total . " €";
```

6. Funciones (php3 a1)

Cómo funciona

- `function nombre($param1, $param2 = valorPorDefecto) { ... return $resultado; }`
- `return` **devuelve** el valor; no se mezcla con `echo`. Quien llama decide si lo imprime.
- Una función no se ejecuta hasta que la llamas.
- Las variables de dentro son locales; para usar una global hay que declarar `global $x;` (o pasarla como parámetro, que es mejor práctica).
- Se pueden tipar los parámetros: `function suma(int $a, int $b): int`.

```
function saludo($nombre) { return "Hola " . $nombre; }
function sumar($a, $b) { return $a + $b; }

echo saludo("Ana") . "<br>";
echo "Suma: " . sumar(3, 4) . "<br>";

function precioFinal($precio, $iva = 21) {
    return $precio * (1 + $iva / 100);
}
echo precioFinal(100) . "<br>";      // 121
echo precioFinal(100, 10) . "<br>";  // 110

function esMayorDeEdad($edad) { return $edad >= 18; }
var_dump(esMayorDeEdad(17));  // bool(false)

function factorial($n) {
```

```
    $f = 1;
    for ($i = $n; $i >= 1; $i--) { $f *= $i; }
    return $f;
}
echo factorial(5);  // 120

$cont = 0;
function incrementar() { global $cont; $cont++; }
incrementar();
echo $cont;  // 1
```

7. Formularios GET y POST (php3 a2 y php4 a1)

Cómo funciona

- `<form method="POST" action="">` envía a la misma página. Los datos llegan en `$_POST["campo"]` (o `$_GET` si el método es GET). La clave es el atributo `name` del input.
- **GET** viaja en la URL (búsquedas, filtros); **POST** va en el cuerpo (login, datos sensibles, altas).
- `isset($_POST["x"])` → ¿existe? `empty($x)` → ¿vacío (“”, 0, null)? `trim()` quita espacios.
- **Siempre** escapa lo que imprimes: `htmlspecialchars($dato)` evita que se ejecute HTML o JavaScript metido por el usuario (XSS).
- Valida en el servidor aunque el HTML tenga `required`.

```
<!-- Formulario POST con validación -->
<form method="POST" action="">
    Nombre: <input type="text" name="nombre"><br>
    Email: <input type="email" name="email"><br>
    <input type="submit" value="Enviar">
</form>

<?php
if (isset($_POST["nombre"], $_POST["email"])) {
    $nombre = trim($_POST["nombre"]);
    $email  = trim($_POST["email"]);

    if (empty($nombre)) {
        echo "Error: falta el nombre.<br>";
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        echo "Error: email no válido.<br>";
    } else {
```

```
        echo "Hola " . htmlspecialchars($nombre);
    }
}
?>

<!-- Formulario GET con select -->
<form method="GET" action="">
    Nombre: <input type="text" name="nombre"><br>
    Edad: <input type="number" name="edad"><br>
    Ciudad:
    <select name="ciudad">
        <option value="Santander" selected>Santander</option>
        <option value="Barcelona">Barcelona</option>
        <option value="Madrid">Madrid</option>
    </select><br>
    <input type="submit" value="Enviar">
</form>

<?php
if (isset($_GET["nombre"], $_GET["edad"], $_GET["ciudad"])) {
    echo "Hola " . htmlspecialchars($_GET["nombre"]) . ", de " .
htmlspecialchars($_GET["ciudad"]);
}
?>
```

8. JSON, filter_var y cadenas (php4)

Cómo funciona

- `file_get_contents($ruta)` lee un archivo local o una URL (la URL necesita `allow_url_fopen` activado).
- `json_decode($texto, true)` convierte JSON en array asociativo (con `true`; sin él, en objetos). Comprueba que salió bien con `is_array()`.
- Los anidados se leen encadenando claves: `$usuario["address"]["city"]`.
- `filter_var($x, FILTER_VALIDATE_EMAIL)` valida email; también `FILTER_VALIDATE_INT`, `FILTER_VALIDATE_URL`.
- `is_numeric($x)` comprueba si es un número.
- Cadenas: `strlen`, `strtoupper`, `strtolower`, `ucfirst`, `str_replace(buscar, reemplazo, texto)`, `substr(texto, inicio, longitud)`, `strpos`, `trim`, `explode`, `implode`, `str_word_count`, `strrev`.

```php
$json = file_get_contents("resources/users.json");
$usuarios = json_decode($json, true);

if (!is_array($usuarios)) {
    echo "No se pudo leer el JSON";
} else {
    echo "<table border='1'><tr><th>ID</th><th>Nombre</th><th>Ciudad</th>
</tr>";
    foreach ($usuarios as $u) {
        echo "<tr><td>" . $u["id"] . "</td>";
        echo "<td>" . htmlspecialchars($u["name"]) . "</td>";
        echo "<td>" . htmlspecialchars($u["address"]["city"]) . "</td></tr>";
    }
    echo "</table>";
}

// Validar una edad
if (isset($_GET["edad"])) {
    echo is_numeric($_GET["edad"]) ? "Edad válida" : "Edad no válida";
}

// Mostrar datos del usuario sin riesgo de XSS
if (isset($_POST["nombre"], $_POST["comentario"])) {
    echo "<p>Nombre: " . htmlspecialchars($_POST["nombre"]) . "</p>";
    echo "<p>Comentario: " . htmlspecialchars($_POST["comentario"]) . "</p>";
}

// Cadenas
echo str_replace("días", "noches", "Buenos días") . "<br>";
$frase = "buenas tardes señor Diego, compañero mío de DAW";
$frase = str_replace("Diego", "Óscar", $frase);
echo $frase . "<br>";
echo substr($frase, 0, 6);  // buenas
```

9. Ejercicios tipo examen

Ejercicios con los mismos temas que las actividades del repositorio, cada uno con su solución. Conviene resolverlos antes de mirar la solución.

Ejercicio 1 · Par/impar y tabla de multiplicar con formulario

Crea un formulario (POST) que pida un número. Al enviarlo, indica si es par o impar y muestra su tabla de multiplicar del 1 al 10. Si no es numérico, muestra un error.

```
<form method="POST">
    Número: <input type="text" name="n">
    <input type="submit" value="Calcular">
</form>
<?php
if (isset($_POST["n"])) {
    if (!is_numeric($_POST["n"])) {
        echo "Error: introduce un número.";
    } else {
        $n = (int) $_POST["n"];
        echo $n % 2 == 0 ? "$n es par<br>" : "$n es impar<br>";
        for ($i = 1; $i <= 10; $i++) {
            echo "$n x $i = " . ($n * $i) . "<br>";
        }
    }
}
?>
```

Ejercicio 2 · Calificación con función y switch

Haz una función `calificacion($nota)` que devuelva «Suspenso» (0–4), «Suficiente» (5), «Bien» (6), «Notable» (7–8), «Sobresaliente» (9–10) o «Nota no válida» si está fuera de 0–10.

```
function calificacion($nota) {
    switch (true) {
        case $nota < 0 || $nota > 10: return "Nota no válida";
        case $nota < 5:  return "Suspenso";
        case $nota == 5: return "Suficiente";
        case $nota == 6: return "Bien";
        case $nota <= 8: return "Notable";
        default:         return "Sobresaliente";
    }
}
echo calificacion(7);  // Notable
```

Ejercicio 3 · Login simulado

Formulario con usuario y contraseña. Usuario correcto: `user`; contraseña: `segura`. Muestra «El usuario no coincide», «La contraseña es incorrecta» o «Bienvenido, user».

```
$usuarioOk = "user";
$passOk = "segura";

if (isset($_POST["usuario"], $_POST["pass"])) {
    $u = trim($_POST["usuario"]);
    $p = $_POST["pass"];

    if ($u !== $usuarioOk) {
        echo "El usuario no coincide";
    } elseif ($p !== $passOk) {
        echo "La contraseña es incorrecta";
    } else {
        echo "Bienvenido, " . htmlspecialchars($u);
    }
}
```

Ejercicio 4 · Carrito con array multidimensional

Dado un array de productos con `nombre`, `precio` y `cantidad`, muestra cada línea con su subtotal, el total sin IVA, el IVA (21%) y el total con IVA (con 2 decimales).

```
$carrito = [
    ["nombre" => "Leche",  "precio" => 1.50, "cantidad" => 6],
    ["nombre" => "Huevos", "precio" => 3.20, "cantidad" => 2],
    ["nombre" => "Pan",    "precio" => 1.10, "cantidad" => 3],
];

$subtotal = 0;
foreach ($carrito as $p) {
    $linea = $p["precio"] * $p["cantidad"];
    echo $p["nombre"] . ": " . number_format($linea, 2) . " €<br>";
    $subtotal += $linea;
}
$iva = $subtotal * 0.21;
echo "Subtotal: " . number_format($subtotal, 2) . " €<br>";
echo "IVA: " . number_format($iva, 2) . " €<br>";
echo "Total: " . number_format($subtotal + $iva, 2) . " €";
```

Ejercicio 5 · Funciones: primo, factorial y Fibonacci

Crea `esPrimo($n)` (devuelve bool), `factorial($n)` y `fibonacci($n)` (devuelve array con los $n primeros términos).

```
function esPrimo($n) {
    if ($n < 2) return false;
    for ($i = 2; $i * $i <= $n; $i++) {
        if ($n % $i == 0) return false;
    }
    return true;
}

function fibonacci($n) {
    $f = [];
    for ($i = 0; $i < $n; $i++) {
        $f[] = $i < 2 ? $i : $f[$i - 1] + $f[$i - 2];
    }
    return $f;
}

var_dump(esPrimo(17));                 // true
echo implode(", ", fibonacci(8));      // 0, 1, 1, 2, 3, 5, 8, 13
```

Ejercicio 6 · Formulario de contacto con validación

Campos: nombre, email y mensaje (mínimo 10 caracteres). Valida todo en el servidor, muestra **todos** los errores a la vez y, si es correcto, muestra los datos escapados.

```
$errores = [];
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $nombre  = trim($_POST["nombre"] ?? "");
    $email   = trim($_POST["email"] ?? "");
    $mensaje = trim($_POST["mensaje"] ?? "");

    if ($nombre === "") $errores[] = "Falta el nombre.";
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) $errores[] = "Email no
válido.";
    if (strlen($mensaje) < 10) $errores[] = "El mensaje debe tener al menos
10 caracteres.";

    if (empty($errores)) {
        echo "Gracias, " . htmlspecialchars($nombre) . ".<br>";
        echo "Mensaje: " . htmlspecialchars($mensaje);
    } else {
        foreach ($errores as $e) echo $e . "<br>";
    }
}
```

Ejercicio 7 · JSON: tabla filtrada por ciudad

Lee `resources/users.json` y muestra una tabla con id, nombre, email y ciudad. Añade un formulario GET con un campo «ciudad» que filtre la tabla (si está vacío, muestra todos).

```
$usuarios = json_decode(file_get_contents("resources/users.json"), true);
$ciudad = trim($_GET["ciudad"] ?? "");
?>
<form method="GET">
    Ciudad: <input type="text" name="ciudad" value="<?=
htmlspecialchars($ciudad) ?>">
    <input type="submit" value="Filtrar">
</form>
<table border="1">
<?php
foreach ($usuarios as $u) {
    $c = $u["address"]["city"];
    if ($ciudad === "" || stripos($c, $ciudad) !== false) {
        echo "<tr><td>{$u['id']}</td><td>" . htmlspecialchars($u["name"]) . "
</td>";
        echo "<td>" . htmlspecialchars($u["email"]) . "</td><td>" .
htmlspecialchars($c) . "</td></tr>";
    }
}
?>
</table>
```

Ejercicio 8 · Cadenas: palíndromo y estadísticas

Dada una frase, muestra su longitud, número de palabras, en mayúsculas, invertida y si es palíndromo (ignorando espacios y mayúsculas).

```
$frase = "Anita lava la tina";

echo strlen($frase) . " caracteres<br>";
echo str_word_count($frase) . " palabras<br>";
echo strtoupper($frase) . "<br>";
echo strrev($frase) . "<br>";

$limpia = strtolower(str_replace(" ", "", $frase));
echo $limpia === strrev($limpia) ? "Es palíndromo" : "No es palíndromo";
```

Ejercicio 9 · Arrays: estadísticas de notas

Dado `$notas = [7, 3, 9, 5, 10, 4];` calcula la media, la nota máxima y mínima, cuántas son aprobadas y muéstralas ordenadas de mayor a menor.

```
$notas = [7, 3, 9, 5, 10, 4];

$media = array_sum($notas) / count($notas);
echo "Media: " . round($media, 2) . "<br>";
echo "Máxima: " . max($notas) . " · Mínima: " . min($notas) . "<br>";

$aprobadas = array_filter($notas, fn($n) => $n >= 5);
echo "Aprobadas: " . count($aprobadas) . "<br>";

rsort($notas);
echo implode(", ", $notas);   // 10, 9, 7, 5, 4, 3
```

Ejercicio 10 · Validar datos con filter_var

Valida con un formulario: edad (entero entre 1 y 120), email y web (URL). Muestra qué campo falla.

```
$edad = filter_var($_POST["edad"] ?? "", FILTER_VALIDATE_INT,
        ["options" => ["min_range" => 1, "max_range" => 120]]);
$email = filter_var($_POST["email"] ?? "", FILTER_VALIDATE_EMAIL);
$web   = filter_var($_POST["web"] ?? "", FILTER_VALIDATE_URL);

if ($edad === false) echo "Edad no válida<br>";
if ($email === false) echo "Email no válido<br>";
if ($web === false) echo "URL no válida<br>";
```

Ejercicio 11 (extra) · Contador de visitas con sesión

Tema que aún no aparece en el repositorio, pero es probable que sea el siguiente. Muestra cuántas veces ha recargado el usuario la página.

```
<?php
session_start();   // SIEMPRE antes de cualquier salida HTML
$_SESSION["visitas"] = ($_SESSION["visitas"] ?? 0) + 1;
echo "Has visitado esta página " . $_SESSION["visitas"] . " veces";
?>
```

10. Funciones que faltaban: sanitizar, validar y más

Sanitizar es limpiar el dato que llega del usuario; **validar** es comprobar que cumple el formato esperado. Regla de oro: validar al recibir y escapar al imprimir con `htmlspecialchars`.

10.1 Sanitizar

Función	Qué hace
`trim($s)`	Quita espacios y saltos de línea al principio y al final
`htmlspecialchars($s, ENT_QUOTES, "UTF-8")`	Convierte `< > & " '` en entidades HTML; evita XSS al imprimir
`strip_tags($s)`	Elimina las etiquetas HTML y PHP del texto
`filter_var($s, FILTER_SANITIZE_FULL_SPECIAL_CHARS)`	Equivale a `htmlspecialchars`
`filter_var($s, FILTER_SANITIZE_EMAIL)`	Elimina los caracteres no permitidos en un email
`filter_var($s, FILTER_SANITIZE_NUMBER_INT)`	Deja solo dígitos, `+` y `-`
`filter_var($s, FILTER_SANITIZE_URL)`	Elimina los caracteres no permitidos en una URL
`intval($s)` o `(int) $s`	Fuerza un entero (`floatval` para decimales)

Ojo: `FILTER_SANITIZE_STRING` está obsoleto desde PHP 8.1; se usa `htmlspecialchars` o `strip_tags`. `addslashes` no protege de la inyección SQL: para eso están las consultas preparadas (10.3).

10.2 Validar

Función	Qué comprueba
`filter_var($x, FILTER_VALIDATE_EMAIL)`	Email válido (devuelve el email o `false`)
`filter_var($x, FILTER_VALIDATE_INT, [...])`	Entero, con rango opcional (`min_range`, `max_range`)

Función	Qué comprueba
`filter_var($x, FILTER_VALIDATE_FLOAT)`	Número decimal
`filter_var($x, FILTER_VALIDATE_URL)`	URL válida
`filter_input(INPUT_POST, "email", FILTER_VALIDATE_EMAIL)`	Lee de POST/GET y filtra a la vez: `null` si no llega, `false` si no es válido
`is_numeric($x)` · `ctype_digit($x)` · `ctype_alpha($x)`	Número, solo dígitos, solo letras
`preg_match("/^[A-Za-z]+$/", $s)`	Expresión regular (devuelve 1 si coincide, 0 si no)
`strlen($s)` · `mb_strlen($s)`	Longitud (`mb_` si hay tildes o ñ)
`isset($x)` · `empty($x)`	Existe y no es null / vacío (incluye "0")

Ejemplo completo: sanitizar y validar un formulario.

```
function limpiar($dato) {
    return htmlspecialchars(trim($dato), ENT_QUOTES, "UTF-8");
}

$nombre = limpiar($_POST["nombre"] ?? "");
$email  = filter_var(trim($_POST["email"] ?? ""), FILTER_SANITIZE_EMAIL);
$edad   = filter_var($_POST["edad"] ?? "", FILTER_VALIDATE_INT,
          ["options" => ["min_range" => 1, "max_range" => 120]]);

$errores = [];
if ($nombre === "") $errores[] = "El nombre es obligatorio.";
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) $errores[] = "Email no
válido.";
if ($edad === false) $errores[] = "La edad debe ser un entero entre 1 y
120.";

if (empty($errores)) {
    echo "Hola " . $nombre;
} else {
    foreach ($errores as $e) echo $e . "<br>";
}
```

10.3 Contraseñas y base de datos

```
// Registro: se guarda solo el hash, nunca la contraseña en claro
$hash = password_hash($_POST["pass"], PASSWORD_DEFAULT);

// Login: se compara contra el hash guardado
if (password_verify($_POST["pass"], $hashGuardado)) {
    echo "Contraseña correcta";
}

// Consulta preparada con PDO (evita la inyección SQL)
$pdo = new PDO("mysql:host=localhost;dbname=demo;charset=utf8mb4", "root",
"");
$stmt = $pdo->prepare("SELECT * FROM usuarios WHERE email = :email");
$stmt->execute([":email" => $email]);
$usuario = $stmt->fetch(PDO::FETCH_ASSOC);
```

10.4 Cadenas, arrays, fechas y matemáticas

- **Buscar en cadenas**: `str_contains($t, $b)` · `str_starts_with` · `str_ends_with` (PHP 8) · `strpos($t, $b)` devuelve la posición o `false`, así que se compara con `!== false` · `substr_count`
- **Dar formato**: `ucwords` · `lcfirst` · `mb_strtoupper` (con tildes) · `str_pad("7", 3, "0", STR_PAD_LEFT)` → `007` · `str_repeat` · `sprintf("%.2f", $n)` · `number_format($n, 2, ",", ".")` · `nl2br`
- **Trocear**: `explode(",", $s)` · `implode(", ", $a)` · `str_split` · `ltrim` / `rtrim` · `strcmp`
- **Añadir y quitar en arrays**: `array_push` · `array_pop` · `array_shift` · `array_unshift` · `array_splice`
- **Combinar y cortar**: `array_merge` · `array_slice` · `array_unique` · `array_reverse` · `array_combine` · `range(1, 5)`
- **Buscar en arrays**: `in_array` · `array_search` · `array_key_exists` · `array_keys` · `array_column`
- **Ordenar**: `sort` · `rsort` · `asort` · `arsort` · `ksort` · `krsort` · `usort` (comparador propio con `<=>`)
- **Fechas**: `date("d/m/Y H:i:s")` · `time()` · `strtotime("2026-10-19")` · `checkdate($m, $d, $a)` · `date_default_timezone_set("Europe/Madrid")` · clase `DateTime` (`->format()`, `->diff()`)
- **Matemáticas**: `pow` / `**` · `sqrt` · `intdiv` · `fmod` · `mt_rand` / `random_int` · `pi()` · `round($n, 2)`

- **Constantes**: `define("IVA", 21);` o `const IVA = 21;`

```
$alumnos = [
    ["nombre" => "Ana",  "nota" => 8],
    ["nombre" => "Luis", "nota" => 5],
    ["nombre" => "Eva",  "nota" => 9],
];

usort($alumnos, fn($a, $b) => $b["nota"] <=> $a["nota"]);   // mayor a menor
echo implode(", ", array_column($alumnos, "nombre"));       // Eva, Ana, Luis

$media = array_sum(array_column($alumnos, "nota")) / count($alumnos);
echo round($media, 2);                                       // 7.33
```

10.5 Sesiones, cookies, cabeceras y ficheros

- **Superglobales**: `$_GET` · `$_POST` · `$_REQUEST` · `$_SERVER` · `$_COOKIE` · `$_SESSION` · `$_FILES`
- **Sesiones**: `session_start()` (siempre antes de imprimir nada) · `$_SESSION["usuario"] = ...` · `unset($_SESSION["x"])` · `session_destroy()`
- **Cookies**: `setcookie("tema", "oscuro", time() + 3600 * 24 * 30, "/")`; se leen con `$_COOKIE["tema"]`
- **Redirigir**: `header("Location: panel.php"); exit;` (antes de cualquier salida)
- **Incluir archivos**: `include` · `require` · `include_once` · `require_once "conexion.php";`
- **Ficheros y JSON**: `file_get_contents` · `file_put_contents` · `file_exists` · `json_encode($a, JSON_PRETTY_PRINT)` · `json_decode($j, true)`
- **Depurar y comprobar tipos**: `var_dump` · `print_r` (dentro de `<pre>`) · `die()` / `exit` · `is_array` · `is_string` · `is_null` · `unset`

```
// login.php
session_start();
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $usuario = trim($_POST["usuario"] ?? "");
    $pass    = $_POST["pass"] ?? "";

    if ($usuario === "user" && $pass === "segura") {
        $_SESSION["usuario"] = $usuario;
        header("Location: panel.php");
        exit;
    }
    echo "Credenciales incorrectas";
}
```

```
// panel.php
session_start();
if (!isset($_SESSION["usuario"])) {
    header("Location: login.php");
    exit;
}
echo "Bienvenido, " . htmlspecialchars($_SESSION["usuario"]);

// logout.php
session_start();
session_destroy();
header("Location: login.php");
exit;
```

11. Chuleta final

Strings: `strlen` · `strtoupper` / `strtolower` · `ucfirst` · `trim` · `str_replace` · `substr` · `strpos` · `explode` / `implode` · `strrev` · `str_word_count`

Arrays: `count` · `array_sum` · `max` / `min` · `sort` / `rsort` / `asort` / `ksort` · `in_array` · `array_push` / `[]` · `array_filter` · `array_map` · `array_keys` / `array_values` · `array_search`

Matemáticas: `round` · `floor` · `ceil` · `abs` · `rand(1, 10)` · `number_format($n, 2, ",", ".")`

Formularios y seguridad: `isset` · `empty` · `$_GET` / `$_POST` · `htmlspecialchars` · `filter_var` · `is_numeric` · `$_SERVER["REQUEST_METHOD"]`

Sanitizar y validar: `trim` · `htmlspecialchars` · `strip_tags` · `filter_var` (`FILTER_SANITIZE_*` / `FILTER_VALIDATE_*`) · `filter_input` · `intval` · `preg_match` · `password_hash` / `password_verify` · `PDO::prepare` / `execute`

Sesiones y otros: `session_start` · `$_SESSION` · `setcookie` · `header("Location: ...")` · `require_once` · `date` · `strtotime` · `usort` · `array_column`

Errores que más puntos cuestan

- Olvidar `;` o `.` al concatenar.
- Usar una variable con otro nombre del que declaraste (`$user` vs `$usuario`).
- Olvidar `break` en un `switch` o cerrar la llave.
- Bucles que nunca terminan (la variable de control no cambia).
- Escribir comentarios `//` fuera de las etiquetas `<?php ?>`.
- No escapar con `htmlspecialchars` lo que viene del usuario.

- Confundir `=` (asignar) con `==` / `===` (comparar).

12. Plan de repaso hasta el 19

- **Dom 11 y Lun 12**: bloques 2–3 (variables, condicionales). Rehacer las actividades de php1 y php2 a1 con las versiones corregidas; ejercicios 1–3.
- **Mar 13**: bucles y arrays; ejercicios 4 y 9.
- **Mié 14**: funciones; ejercicio 5.
- **Jue 15 y Vie 16**: formularios y validación; ejercicios 6 y 10.
- **Sáb 17**: JSON y cadenas; ejercicios 7 y 8. Repasar también sesiones (ejercicio 11).
- **Dom 18**: simulacro: 3 ejercicios al azar en 1 hora, sin consultar nada.